INP - ENSEIRB-MATMECA – TELECOM 2A

ET8 PG219 : Développement d'applications pour terminaux mobiles

Projet Genoa : Application de gestion d'arbre généalogique intelligent 🌳

🎯 Introduction

Le projet doit être réalisé en **binôme** mais chacun des membres sera **évalué individuellement** donc les tâches doivent être clairement réparties.

Il a pour but de développer une application de gestion avancée d'arbre généalogique familial. Cette application se nommera **Genoa**.

Genoa a pour objectif de permettre à une famille de construire, visualiser et analyser dynamiquement son arbre généalogique à travers une interface interactive et sécurisée sur appareil mobile.

Pour la réaliser, il sera nécessaire de mettre en œuvre une architecture client-serveur.

L'application comportera :

- Une partie serveur (API REST)
- Une application ReactNative

Le code devra être implémenté avec **ReactNative** et une base de données de votre choix.

📦 Rendu

Le rendu final du projet est :

- Un lien vers un dépôt GitHub intitulé **Genoa**
- Un lien vers l'application déployée en ligne

Le travail est à déposer dans ce document partagé avant le **2 avril** :

https://docs.google.com/spreadsheets/d/1nT4DFv47Gu-6cl87LkAltAuvR0Q0Gwgt13SSgBtzWSc/edit?usp=sharing

Le dépôt devra contenir :

- Le code source client/serveur
- Les scripts de création de la base de données
- Un fichier README.md listant les **fonctionnalités** implémentées pour chacune d'elle **l'auteur** devra être indiqué en cohérence avec les commits sur github.

Chaque binôme disposera de **15 minutes** pour présenter son projet et réaliser une démonstration vendredi après-midi **3 avril**.

🧩 Fonctionnalités

🔐 1️⃣ Inscription & Authentification

L'application doit permettre :

- L'inscription des utilisateurs
- L'authentification avec génération d'un JWT valide 24h

Les informations requises à l'inscription :

- e-mail (unique dans la base)
- mot de passe

Chaque inscription doit être validée par l'administrateur.

Le premier inscrit devient administrateur.

👤 2️⃣ Gestion des utilisateurs (Administration)

Une page d’administration doit permettre aux super-utilisateurs de :

- Valider l’inscription des autres utilisateurs
- Supprimer un compte utilisateur
- Partager les droits d’administration
- Modifier un compte utilisateur
- Créer un compte pour un tiers

🌳3️⃣ Gestion des membres (nœuds de l’arbre)

Un utilisateur peut ajouter un membre de la famille comprenant les attributs suivants :

- Id obligatoire
- Nom
- Prénom
- Sexe
- Photo
- Date de naissance
- Date de décès
- Profession(s)
- Coordonnées (adresse(s), téléphone, e-mail(s))
- Informations complémentaires
    - Privée
    - Publique

Un éditeur doit pouvoir :

- Modifier un membre
- Supprimer un membre

Les données privées doivent être visibles uniquement par les éditeurs.

🔗4️⃣ Gestion des relations familiales

Un utilisateur peut ajouter les relations suivantes entre deux membres :

Couple / Enfant

- Id du couple
- Id de l'enfant
- Biologique (ou adopté)

Couples

- Id des deux membres
- Date d'union facultative
- Date de séparation facultative

Les relations doivent être cohérentes (pas de cycle impossible, validation logique minimale).

📊5️⃣ Visualisation de l'arbre généalogique

Il doit être possible de visualiser l'arbre sous forme graphique interactive.

Contraintes :

- Affichage dynamique (dagre, elkjs, d3-dag)
- Zoom / déplacement
- Sélection d'un membre pour afficher ses détails
- Mise à jour en temps réel après modification

Par défaut :

- Seuls les parents et enfants de l'utilisateur identifié sont affichés
- Possibilité d'étendre ou limiter la visualisation
  - aux enfants, aux parents, la fratrie, les descendants ou les ascendants (avec ou sans conjoints)
  - à certains champs (ex : nom et prénom uniquement)

🔍6️⃣ Recherche & navigation

L'application doit permettre :

- Recherche par nom ou prénom
- Navigation ascendante (ancêtres)
- Navigation descendante (descendants)
- Visualisation de la fratrie

📈 7 Statistiques familiales

Une page statistique doit afficher :

- Nombre total de membres
- Nombre d'hommes / femmes
- Espérance de vie moyenne
- Nombre moyen d'enfants par génération
- Nombre de générations

Ces statistiques doivent être calculées dynamiquement.

🛡️ 8 Gestion des droits & confidentialité

Il existe 3 statuts pour les membres donnant accès à des droits différents :

- Administrateur : dispose de tous les droits cf gestion des utilisateurs + éditeur
- Éditeur : peut lire et écrire mais ne dispose pas des droits de gestion utilisateurs.
- Lecteur : peut simplement lire

Les données privées ne sont visibles que par :

- Le créateur
- Les administrateurs

Un système de **permissions** et de **concurrence d'accès** à l'aide de ***verrous*** et de ***synchronisation en temps réel (Socket.IO)*** doit être mis en place.

🧱 Contraintes techniques

Le projet doit inclure :

- Architecture propre (séparation API / UI)
- Middleware d'authentification
- Protection des routes
- Validation des données

- Gestion des variables d’environnement

⭐ Extensions possibles

Chaque extension donnera lieu à des points supplémentaires.

Déploiement

- Exécution en code natif sur iOS ou Androïd

Affichage

- Couleurs dynamiques (par âge, génération, sexe)
- Mode chronologique (timeline familiale)
- Visualisation cartographique (Leaflet) pour voir la dispersion de la famille sur une carte.

Fonctionnalités avancées

- Recherche avancée multicritère
- Comparaison entre deux branches
- Export / Import JSON
- Export PDF de l’arbre
- Historique des modifications (log admin)
- **Undo / Redo**
- Authentification déléguée sur github / google …
- Multi-langues
- Système de commentaires sur un membre
- Génération automatique d’un arbre simplifié partageable

🧠 Points pédagogiques mobilisés

Ce projet mobilise :

- Authentification JWT
- Gestion des droits
- Relations complexes en base de données
- Modélisation de graphes
- Visualisation de données (D3.js)

- Sécurité & permissions
- Gestion d’état avancée
- Manipulation de données relationnelles

🤖 Limitation du recours à l’IA et au web

- Ne recopiez pas un code que vous ne comprenez pas ou que vous ne sauriez pas réécrire par cœur (⚠️ vous serez interrogés sur votre code).

🎯 Objectif pédagogique final

À l’issue du projet, l’étudiant doit démontrer sa capacité à :

- Concevoir une architecture complète
- Modéliser des relations complexes
- Implémenter une visualisation dynamique
- Gérer des permissions avancées
- Sécuriser une application fullstack pour appareil mobile